

Helix OTA

Helix OTA — Database Schema Specification

Document ID: HELOTA-DB-001 **Version:** 1.0.0 **Status:** Active **Last Updated:** 2026-03-05 **Constitution Reference:** HelixConstitution v1 §1-§4 **Target Engine:** PostgreSQL 16+

Table of Contents

- [1. Database Overview](#)
 - [2. Enum Types](#)
 - [3. Core Tables](#)
 - [4. Index Strategy](#)
 - [5. Migration Strategy](#)
 - [6. Sample Queries](#)
 - [7. Partitioning Strategy](#)
 - [8. Replication & Backup](#)
-

1. Database Overview

1.1 Engine & Version

Helix OTA uses **PostgreSQL 16+** as its sole relational data store. PostgreSQL was selected for its native UUID support, JSONB indexing, declarative partitioning, row-level security capabilities, and mature ecosystem of tooling for replication and point-in-time recovery. No auxiliary databases (e.g., TimescaleDB) are required for the MVP; native PostgreSQL partitioning satisfies the time-series requirements of the telemetry subsystem.

1.2 Primary Key Convention

All primary keys use the **UUID v4** type (UUID in PostgreSQL). This provides:

- **Globally unique identifiers** — no collision risk across shards or environments.
- **Client-side generation** — the application layer generates UUIDs before INSERT, eliminating the need for sequence-based ID generation and reducing round-trips.
- **Non-sequential** — prevents enumeration attacks on public-facing APIs.

```
-- Application-side generation using pgcrypto as a fallback
CREATE EXTENSION IF NOT EXISTS "pgcrypto";
-- Default UUID generation: gen_random_uuid() (built-in
    since PG 13)
```

1.3 Timestamp Conventions

All timestamp columns use TIMESTAMPTZ (timestamp with time zone) stored in UTC. The application layer is responsible for converting to local time for display purposes.

Column Pattern	Type	Default	Description
created_at	TIMESTAMPTZ	NOW()	Row creation time; never modified
updated_at	TIMESTAMPTZ	NOW()	Last modification time; updated via trigger
deleted_at	TIMESTAMPTZ	NULL	Soft-delete timestamp; NULL = not deleted
*_at (domain)	TIMESTAMPTZ	NULL	Domain-specific event timestamps

An updated_at auto-update trigger is installed globally:

```
CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Every table with an `updated_at` column has the trigger:

```
CREATE TRIGGER trg_{table_name}_updated_at
  BEFORE UPDATE ON {table_name}
  FOR EACH ROW
  EXECUTE FUNCTION update_updated_at_column();
```

1.4 Soft Delete Pattern

Tables that support soft deletion include a `deleted_at` `TIMESTAMPZ` column. Soft-deleted rows have `deleted_at` set to the deletion timestamp; active rows have `deleted_at IS NULL`.

Query convention: All application queries **MUST** include `WHERE deleted_at IS NULL` when accessing soft-deletable tables. This is enforced through:

1. **Repository layer** — Go repository structs include the filter by default.
2. **Views (optional)** — For complex reporting queries, updatable views pre-filter active rows.
3. **Partial indexes** — Unique constraints and common query indexes include `WHERE deleted_at IS NULL` to exclude soft-deleted rows.

Hard delete policy: Rows are permanently purged after 90 days by a scheduled maintenance job. This ensures GDPR compliance while preserving audit history within the retention window.

1.5 Naming Conventions

Element	Convention	Example
Table names	snake_case, plural	device_groups
Column names	snake_case	hardware_model
Primary keys	id (UUID)	id UUID PRIMARY KEY
Foreign keys	{referenced_table_singular}_id	artifact_id UUID REFERENCES
Index names	idx_{table}_{columns}	idx_devices_group_id
Unique index names	uidx_{table}_{columns}	uidx_devices_device_id
Enum types	snake_case with _enum suffix	device_status_enum

Element	Convention	Example
Check constraints	chk_{table}_{description}	chk_rollouts_target_pct

1.6 Connection & Pooling

The Go application uses pgxpool from the jackc/pgx driver with the following pool configuration:

Parameter	Development	Production
max_conns	10	50
min_conns	2	10
max_conn_idle	30m	15m
max_conn_lifetime	1h	30m
health_check_period	30s	15s

2. Enum Types

All enum types are defined as PostgreSQL native ENUM types for type safety, storage efficiency (4 bytes), and constraint enforcement at the database level.

```
-- =====
-- Device status: tracks the real-time connectivity/state of
-- a device
-- =====
CREATE TYPE device_status_enum AS ENUM (
    'online',    -- Device is connected and responsive
    'offline',   -- Device has not checked in within the
                 -- timeout window
    'updating',  -- Device is actively downloading or
                 -- installing an update
    'error'      -- Device has reported an error and
                 -- requires attention
);

-- =====
-- Storage backend: identifies where artifact binary data is
-- persisted
-- =====
CREATE TYPE storage_backend_enum AS ENUM (
    's3',        -- AWS S3 or S3-compatible API (production)
    'local',     -- Local filesystem (development only)
    'minio'      -- Self-hosted MinIO (staging/production)
);

-- =====
```

```

-- Upload status: tracks the artifact upload and validation
-- pipeline
-- =====
CREATE TYPE upload_status_enum AS ENUM (
    'uploading',    -- Binary data is being streamed to
                    storage
    'validating',   -- Validation chain is executing (hash/
                    signature/structure/compat)
    'ready',        -- All validations passed; artifact is
                    eligible for rollouts
    'failed'        -- Validation or upload failed; artifact
                    is not usable
);

-- =====
-- Validation step: the four-stage artifact validation chain
-- =====
CREATE TYPE validation_step_enum AS ENUM (
    'hash_check',   -- SHA-256 integrity
                    verification
    'signature_check', -- RSA-4096 signature
                    verification
    'structure_check', -- OTA package structure
                    conformance
    'compatibility_check' -- Hardware/version
                    compatibility
);

-- =====
-- Validation status: outcome of a single validation step
-- =====
CREATE TYPE validation_status_enum AS ENUM (
    'passed',       -- Step succeeded
    'failed',       -- Step failed (blocks artifact from
                    becoming ready)
    'skipped'       -- Step was skipped (e.g., signature_check
                    in dev mode)
);

-- =====
-- Rollout strategy: controls how the rollout progresses
-- =====
CREATE TYPE rollout_strategy_enum AS ENUM (
    'instant',      -- 0→100% immediately (emergency patches)
    'canary',       -- Small initial cohort (1-5%) then auto-
                    advance
    'gradual'       -- Configurable staged progression with
                    explicit stages
);

-- =====
-- Rollout status: lifecycle state of a rollout
-- =====
CREATE TYPE rollout_status_enum AS ENUM (

```

```

'draft',          -- Created but not yet started
'paused',         -- Temporarily halted; can be resumed
'active',         -- Currently deploying to devices
'completed',      -- Target percentage reached; all
                   devices processed
'halted',         -- Manually stopped due to issues; no
                   auto-resume
'rolled_back'     -- Rollback initiated; reverting devices
);

-- =====
-- Rollout stage status: lifecycle of a single stage in a
-- gradual rollout
-- =====
CREATE TYPE rollout_stage_status_enum AS ENUM (
    'pending',      -- Stage has not yet started
    'active',       -- Stage is currently deploying
    'completed',    -- Stage target percentage reached
    'failed'        -- Stage failed health check; rollout may
                   be paused
);

-- =====
-- Device update status: per-device update lifecycle state
-- machine
-- =====
CREATE TYPE device_update_status_enum AS ENUM (
    'pending',      -- Update assigned; awaiting device
                   acknowledgment
    'downloading',  -- Device is downloading the artifact
    'verifying',    -- Device is verifying hash and
                   signature
    'installing',   -- Device is applying the update
                   (update_engine)
    'rebooting',    -- Device is rebooting into the new
                   partition
    'committing',   -- Device is committing the new slot
                   (marking boot successful)
    'succeeded',    -- Update fully applied and committed
    'failed',       -- Update failed at some stage
    'rolled_back'   -- Device rolled back to the previous
                   partition
);

-- =====
-- Telemetry event type: categorizes device-reported events
-- =====
CREATE TYPE telemetry_event_type_enum AS ENUM (
    'check',        -- Device checked for updates
    'download_start', -- Device began downloading
    'download_progress', -- Download progress report
                   (periodic)
    'download_complete', -- Download finished successfully

```

```

        'verify_start',      -- Hash/signature verification
                               started
        'verify_complete',  -- Verification completed
        'install_start',    -- Installation began
                               (update_engine invoke)
        'install_complete', -- Installation completed
        'reboot',           -- Device is rebooting
        'commit',           -- New slot committed as active
        'failure',          -- An error occurred
        'rollback'          -- Device rolled back to previous
                               version
    );

-- =====
-- User role: RBAC role for dashboard users
-- =====
CREATE TYPE user_role_enum AS ENUM (
    'admin',
        -- Full access: manage artifacts, rollouts,
        devices, users
    'operator', -- Operational: manage rollouts, view
        devices/artifacts
    'viewer'    -- Read-only: view dashboards, device
        status, telemetry
);

```

3. Core Tables

3.1 device_groups — Logical Device Groupings

Device groups enable targeted rollouts. A group is defined by a set of filter rules expressed as JSONB. For MVP, groups are simple (e.g., “all RK3588 devices”), but the JSONB filter_rules column supports complex future rules.

```

CREATE TABLE device_groups (
    id          UUID          PRIMARY KEY DEFAULT
        gen_random_uuid(),
    name        VARCHAR(128) NOT NULL,
    description TEXT,
    filter_rules JSONB        NOT NULL DEFAULT '{}',
    created_at  TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
    updated_at  TIMESTAMPTZ   NOT NULL DEFAULT NOW(),

    CONSTRAINT chk_device_groups_name_length
        CHECK (length(trim(name)) >= 1)
);

```

```

COMMENT ON TABLE device_groups IS 'Logical device groupings
used for targeted rollout deployment';
COMMENT ON COLUMN device_groups.filter_rules IS 'JSONB
filter rules for automatic device assignment.
Example: {"hardware_model": "rk3588_opi5max",
"os_version": {"gte": "15.0.0"}}';

```

Filter Rules Schema (JSONB):

```

{
  "hardware_model": "rk3588_opi5max",
  "os_type": "android",
  "os_version": { "gte": "15.0.0", "lt": "16.0.0" },
  "tags": ["production", "us-east"]
}

```

3.2 devices — Device Registry

The central registry of all managed devices. Each device has a unique `device_id` (typically derived from hardware serial or a provisioning identifier) and is associated with a device group for rollout targeting.

```

CREATE TABLE devices (
  id                                UUID                                PRIMARY
    KEY DEFAULT gen_random_uuid(),
  device_id                        VARCHAR(256)                        NOT NULL,
  hardware_model                   VARCHAR(128)                        NOT NULL,
  os_type                          VARCHAR(64)                         NOT NULL,
  os_version                       VARCHAR(64)                         NOT NULL,
  current_version                  VARCHAR(64)                         NOT NULL,
  target_version                   VARCHAR(64),
  group_id                        UUID                                NOT NULL
    REFERENCES device_groups(id) ON DELETE RESTRICT,
  mtls_cert_fingerprint            VARCHAR(128),
  ip_address                       INET,
  last_seen_at                    TIMESTAMPTZ,
  status                           device_status_enum                NOT NULL
    DEFAULT 'offline',
  created_at                       TIMESTAMPTZ                        NOT NULL
    DEFAULT NOW(),
  updated_at                      TIMESTAMPTZ                        NOT NULL
    DEFAULT NOW(),
  deleted_at                      TIMESTAMPTZ,

  CONSTRAINT uidx_devices_device_id UNIQUE (device_id),
  CONSTRAINT chk_devices_os_type CHECK (os_type IN
    ('android', 'linux', 'windows')),
  CONSTRAINT chk_devices_status CHECK (status IN
    ('online', 'offline', 'updating', 'error'))
);

```



```

COMMENT ON TABLE devices IS
    'Central registry of all managed devices in the OTA
    fleet';
COMMENT ON COLUMN devices.device_id IS 'Unique device
    identifier, typically derived from hardware serial
    or provisioning ID';
COMMENT ON COLUMN devices.mtls_cert_fingerprint IS 'SHA-256
    fingerprint of the device mTLS client certificate';
COMMENT ON COLUMN devices.target_version IS 'The version
    this device is scheduled to update to; NULL if no
    update is pending';
COMMENT ON COLUMN devices.deleted_at IS 'Soft-delete
    timestamp; NULL means the device is active';

```

3.3 users — Dashboard Users

Administrative users who access the Helix OTA Dashboard. Each user has a role-based access level and optional TOTP two-factor authentication.

```

CREATE TABLE users (
    id          UUID          PRIMARY KEY DEFAULT
        gen_random_uuid(),
    username    VARCHAR(128)  NOT NULL,
    email       VARCHAR(256)  NOT NULL,
    password_hash VARCHAR(256) NOT NULL,
    role        user_role_enum NOT NULL DEFAULT
        'viewer',
    totp_secret  VARCHAR(64),
    totp_enabled BOOLEAN      NOT NULL DEFAULT FALSE,
    last_login_at TIMESTAMPTZ,
    created_at   TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    updated_at   TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    deleted_at   TIMESTAMPTZ,

    CONSTRAINT uidx_users_username UNIQUE (username) WHERE
        deleted_at IS NULL,
    CONSTRAINT uidx_users_email UNIQUE (email) WHERE
        deleted_at IS NULL,
    CONSTRAINT chk_users_password_hash CHECK
        (length(password_hash) >= 60),
    CONSTRAINT chk_users_email_format CHECK (email ~* '^[A-
        Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$')
);

COMMENT ON TABLE users IS 'Dashboard administrative users
    with role-based access control';
COMMENT ON COLUMN users.password_hash IS 'bcrypt hash (60
    characters). Never store plaintext passwords.';
COMMENT ON COLUMN users.totp_secret IS 'Base32-encoded TOTP
    secret for 2FA; NULL if 2FA not configured';

```

3.4 artifacts — OTA Update Artifacts

Update artifacts are the binary packages deployed to devices. Each artifact undergoes a multi-stage validation pipeline before becoming eligible for rollouts. The `payload_metadata` JSONB column captures OTA-package-specific information (e.g., partition sizes, update type).

```
CREATE TABLE artifacts (
  id                UUID                PRIMARY
    KEY DEFAULT gen_random_uuid(),
  filename          VARCHAR(512)       NOT NULL,
  version           VARCHAR(64)        NOT NULL,
  os_type           VARCHAR(64)        NOT NULL,
  os_version        VARCHAR(64)        NOT NULL,
  hardware_compatibility JSONB        NOT NULL
    DEFAULT '[]',
  file_size         BIGINT             NOT NULL,
  file_hash_sha256  VARCHAR(128)       NOT NULL,
  signature_hash    VARCHAR(256),
  signature_algorithm VARCHAR(64)      DEFAULT
    'RSA-4096',
  payload_metadata  JSONB             DEFAULT
    '{}',
  storage_path      VARCHAR(1024),
  storage_backend   storage_backend_enum NOT NULL
    DEFAULT 's3',
  upload_status     upload_status_enum NOT NULL
    DEFAULT 'uploading',
  uploaded_by       UUID
    REFERENCES users(id) ON DELETE SET NULL,
  validated_at      TIMESTAMPTZ,
  created_at        TIMESTAMPTZ       NOT NULL
    DEFAULT NOW(),
  updated_at        TIMESTAMPTZ       NOT NULL
    DEFAULT NOW(),
  deleted_at        TIMESTAMPTZ,

  CONSTRAINT chk_artifacts_file_size_positive CHECK
    (file_size > 0),
  CONSTRAINT chk_artifacts_sha256_format CHECK
    (file_hash_sha256 ~ '^[a-f0-9]{64}$'),
  CONSTRAINT chk_artifacts_version_not_empty CHECK
    (length(trim(version)) >= 1)
);

COMMENT ON TABLE artifacts IS 'OTA update artifacts: binary
packages deployed to devices';
COMMENT ON COLUMN artifacts.hardware_compatibility IS
'JSONB array of compatible hardware models.
Example: ["rk3588_opi5max", "rk3588_opi5"]';
```

```

COMMENT ON COLUMN artifacts.payload_metadata IS
    'OTA package metadata. Example: {"update_type":
    "full", "partitions": {"system": 2147483648,
    "vendor": 536870912}, "source_version_min":
    "15.0.0"}';
COMMENT ON COLUMN artifacts.signature_hash IS 'Hex-encoded
    RSA-4096 signature of the artifact payload';
COMMENT ON COLUMN artifacts.storage_path IS 'Object storage
    key (e.g., artifacts/{id}/ota_update.zip)';

```

Hardware Compatibility Schema (JSONB):

```
["rk3588_opi5max", "rk3588_opi5"]
```

Payload Metadata Schema (JSONB):

```

{
    "update_type": "full",
    "source_version_min": "15.0.0",
    "partitions": {
        "system": 2147483648,
        "vendor": 536870912,
        "boot": 67108864
    },
    "care_map_present": true,
    "ab_ota": true
}

```

3.5 artifact_validation_results — Per-Artifact Validation Log

Each artifact passes through a four-stage validation chain. This table records the outcome of each stage, providing a complete audit trail for compliance and debugging.

```

CREATE TABLE artifact_validation_results (
    id                UUID                PRIMARY KEY
        DEFAULT gen_random_uuid(),
    artifact_id       UUID                NOT NULL
        REFERENCES artifacts(id) ON DELETE CASCADE,
    validation_step    validation_step_enum NOT NULL,
    status            validation_status_enum NOT NULL,
    details            JSONB              DEFAULT '{}',
    duration_ms        INTEGER            NOT NULL
        DEFAULT 0,
    created_at         TIMESTAMPTZ        NOT NULL
        DEFAULT NOW(),

    CONSTRAINT chk_artifact_validation_duration CHECK
        (duration_ms >= 0)
);

```

```

COMMENT ON TABLE artifact_validation_results IS 'Per-
artifact validation step results; provides audit
trail for the four-stage validation chain';
COMMENT ON COLUMN artifact_validation_results.details IS
'JSONB details of the validation result. For
failures: {"error": "...", "expected": "...",
"actual": "..."}'. For passes: {"hash_matched":
true, "algorithm": "SHA-256"}';
COMMENT ON COLUMN artifact_validation_results.duration_ms
IS 'Duration of the validation step in
milliseconds';

```

Details Schema (JSONB):

```

// Failure example:
{
  "error": "SHA-256 mismatch",
  "expected": "abc123...",
  "actual": "def456...",
  "algorithm": "SHA-256"
}

// Success example:
{
  "hash_matched": true,
  "algorithm": "SHA-256",
  "computed_hash": "abc123..."
}

```

3.6 rollouts — Phased Deployment Configurations

Rollouts define how an artifact is deployed to a device group. They support three strategies: instant (0→100%), canary (small initial cohort with auto-advance), and gradual (explicit multi-stage progression).

```

CREATE TABLE rollouts (
  id                UUID                PRIMARY
    KEY DEFAULT gen_random_uuid(),
  artifact_id       UUID                NOT NULL
    REFERENCES artifacts(id) ON DELETE RESTRICT,
  name              VARCHAR(256)        NOT NULL,
  description        TEXT,
  strategy           rollout_strategy_enum NOT
    NULL DEFAULT 'gradual',
  status            rollout_status_enum  NOT
    NULL DEFAULT 'draft',
  target_group_id   UUID
    REFERENCES device_groups(id) ON DELETE SET NULL,
  target_percentage  FLOAT              NOT NULL
    DEFAULT 100.0,
  current_percentage FLOAT              NOT NULL
    DEFAULT 0.0,

```

```

auto_rollback_threshold    FLOAT                NOT NULL
    DEFAULT 0.05,
auto_rollback_enabled      BOOLEAN              NOT NULL
    DEFAULT TRUE,
created_by                 UUID
    REFERENCES users(id) ON DELETE SET NULL,
started_at                TIMESTAMPTZ,
completed_at              TIMESTAMPTZ,
created_at                TIMESTAMPTZ          NOT NULL
    DEFAULT NOW(),
updated_at                TIMESTAMPTZ          NOT NULL
    DEFAULT NOW(),

CONSTRAINT chk_rollouts_target_pct CHECK
    (target_percentage > 0 AND target_percentage <=
100),
CONSTRAINT chk_rollouts_current_pct CHECK
    (current_percentage >= 0 AND current_percentage <=
100),
CONSTRAINT chk_rollouts_current_lte_target CHECK
    (current_percentage <= target_percentage),
CONSTRAINT chk_rollouts_rollback_threshold CHECK
    (auto_rollback_threshold > 0 AND
auto_rollback_threshold <= 1),
CONSTRAINT chk_rollouts_name_not_empty CHECK
    (length(trim(name)) >= 1)
);

COMMENT ON TABLE rollouts IS 'Phased deployment
configurations for distributing artifacts to device
groups';
COMMENT ON COLUMN rollouts.auto_rollback_threshold IS
'Failure rate threshold (0.0-1.0) that triggers
automatic rollback. Default: 0.05 (5%)';
COMMENT ON COLUMN rollouts.target_group_id IS
'Target device group; NULL means all devices
matching artifact compatibility';
COMMENT ON COLUMN rollouts.strategy IS
'Deployment strategy: instant=0→100%, canary=small
cohort+auto-advance, gradual=explicit stages';

```

3.7 rollout_stages — Individual Stages in a Gradual Rollout

When a rollout uses the gradual strategy, each phase of the deployment is tracked as a separate stage. This enables fine-grained control over rollout progression and detailed auditing.

```

CREATE TABLE rollout_stages (
    id                UUID                PRIMARY KEY
    DEFAULT gen_random_uuid(),
    rollout_id        UUID                NOT NULL
    REFERENCES rollouts(id) ON DELETE CASCADE,

```

```

stage_number      INTEGER                        NOT NULL,
target_percentage FLOAT                        NOT NULL,
status            rollout_stage_status_enum NOT NULL
                DEFAULT 'pending',
started_at        TIMESTAMPTZ,
completed_at      TIMESTAMPTZ,
created_at        TIMESTAMPTZ                  NOT NULL
                DEFAULT NOW(),

CONSTRAINT chk_rollout_stages_number_positive CHECK
    (stage_number > 0),
CONSTRAINT chk_rollout_stages_target_pct CHECK
    (target_percentage > 0 AND target_percentage <=
    100),
CONSTRAINT uidx_rollout_stages_rollout_stage UNIQUE
    (rollout_id, stage_number)
);

COMMENT ON TABLE rollout_stages IS
    'Individual stages within a gradual rollout,
    tracking per-phase deployment progress';
COMMENT ON COLUMN rollout_stages.stage_number IS '1-based
    stage number; stages execute in sequential order';
COMMENT ON COLUMN rollout_stages.target_percentage IS 'The
    cumulative rollout percentage this stage aims to
    reach';

```

3.8 device_updates — Per-Device Update Tracking

Each device's individual update attempt is tracked here, forming the bridge between a rollout and the actual device-level state machine. A device may have multiple device_updates rows (one per update attempt), enabling full update history.

```

CREATE TABLE device_updates (
    id                UUID
        PRIMARY KEY DEFAULT gen_random_uuid(),
    device_id         UUID                        NOT
        NULL REFERENCES devices(id) ON DELETE CASCADE,
    rollout_id        UUID
        REFERENCES rollouts(id) ON DELETE SET NULL,
    artifact_id       UUID                        NOT
        NULL REFERENCES artifacts(id) ON DELETE RESTRICT,
    status            device_update_status_enum NOT
        NULL DEFAULT 'pending',
    download_progress  FLOAT                      NOT
        NULL DEFAULT 0.0,
    download_bytes_total BIGINT
        DEFAULT 0,
    download_bytes_completed BIGINT
        DEFAULT 0,
    error_message      TEXT,

```

```

error_code          VARCHAR(64),
started_at          TIMESTAMPTZ,
completed_at        TIMESTAMPTZ,
created_at          TIMESTAMPTZ          NOT
NULL DEFAULT NOW(),

CONSTRAINT chk_device_updates_download_progress CHECK
(download_progress >= 0.0 AND download_progress <=
1.0),
CONSTRAINT chk_device_updates_bytes_non_negative CHECK
(download_bytes_total >= 0 AND
download_bytes_completed >= 0),
CONSTRAINT chk_device_updates_bytes_consistency CHECK
(download_bytes_completed <= download_bytes_total)
);

COMMENT ON TABLE device_updates IS 'Per-device update
tracking; records the full lifecycle of each update
attempt';
COMMENT ON COLUMN device_updates.rollout_id IS 'The rollout
that triggered this update; NULL for manual/direct
updates';
COMMENT ON COLUMN device_updates.download_progress IS
'Download progress as a float between 0.0 and 1.0';
COMMENT ON COLUMN device_updates.error_code IS 'Machine-
readable error code (e.g., HASH_MISMATCH,
SIGNATURE_INVALID, INSTALL_FAILED)';
COMMENT ON COLUMN device_updates.error_message IS 'Human-
readable error description for debugging';

```

3.9 telemetry_events — Time-Series Telemetry Data

Telemetry events are the highest-volume table in the system. Devices report events at every stage of the update lifecycle, plus periodic heartbeats. This table is **partitioned by month** using PostgreSQL declarative partitioning (see [Section 7](#)).

```

CREATE TABLE telemetry_events (
    id          UUID          NOT NULL,
    device_id    UUID          NOT NULL,
    event_type   telemetry_event_type_enum NOT NULL,
    payload      JSONB         DEFAULT '{}',
    created_at   TIMESTAMPTZ   NOT NULL
    DEFAULT NOW(),

    CONSTRAINT pk_telemetry_events PRIMARY KEY (id,
        created_at),
    CONSTRAINT fk_telemetry_events_device FOREIGN KEY
        (device_id) REFERENCES devices(id) ON DELETE
        CASCADE
) PARTITION BY RANGE (created_at);

```

```

COMMENT ON TABLE telemetry_events IS 'Time-series telemetry
data from devices; partitioned by month for query
performance and data lifecycle management';
COMMENT ON COLUMN telemetry_events.payload IS
'JSONB payload with event-specific data. Example
for download_progress: {"percent": 45,
"bytes_per_sec": 12500000}';

```

Payload Schema Examples (JSONB):

```

// download_progress event
{
  "percent": 45,
  "bytes_per_sec": 12500000,
  "bytes_downloaded": 1073741824,
  "bytes_total": 2147483648
}

// failure event
{
  "stage": "installing",
  "error_code": "INSTALL_FAILED_VERIFICATION",
  "error_message": "Payload verification failed",
  "slot": "_b"
}

// commit event
{
  "old_version": "15.0.0",
  "new_version": "15.0.1",
  "slot": "_a",
  "boot_successful": true
}

```

3.10 audit_log — Security Audit Trail

Every administrative action is logged for compliance and forensic analysis. The audit log is append-only; rows are never updated or soft-deleted.

```

CREATE TABLE audit_log (
  id          UUID          PRIMARY KEY DEFAULT
    gen_random_uuid(),
  user_id     UUID          REFERENCES users(id) ON
    DELETE SET NULL,
  action      VARCHAR(128) NOT NULL,
  resource_type VARCHAR(64) NOT NULL,
  resource_id UUID,
  details     JSONB         DEFAULT '{}',
  ip_address  INET,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```



```
COMMENT ON TABLE audit_log IS 'Append-only security audit
trail; records all administrative actions for
compliance and forensics';
COMMENT ON COLUMN audit_log.user_id IS 'The user who
performed the action; NULL for system-initiated
actions';
COMMENT ON COLUMN audit_log.action IS 'Action verb: create,
update, delete, login, login_failed, pause, resume,
halt, rollback';
COMMENT ON COLUMN audit_log.resource_type IS 'Type of
resource affected: artifact, rollout, device, user,
device_group';
COMMENT ON COLUMN audit_log.details IS
'JSONB details of the action. Example: {"field":
"status", "old_value": "active", "new_value":
"paused"}';
```

4. Index Strategy

The index strategy is designed around the actual query patterns identified in the application layer. Each index has a documented rationale. Over-indexing is avoided to prevent write performance degradation.

4.1 Primary Key Indexes (Automatic)

PostgreSQL automatically creates unique B-tree indexes for primary keys. No additional action required.

Table	Column	Type
device_groups	id	B-tree (auto)
devices	id	B-tree (auto)
users	id	B-tree (auto)
artifacts	id	B-tree (auto)
artifact_validation_results	id	B-tree (auto)
rollouts	id	B-tree (auto)
rollout_stages	id	B-tree (auto)
device_updates	id	B-tree (auto)
audit_log	id	B-tree (auto)

Note: telemetry_events primary key is (id, created_at) to align with partition key requirements.

4.2 Unique Constraint Indexes

```
-- devices: device_id must be globally unique (business key)
CREATE UNIQUE INDEX uidx_devices_device_id ON devices
    (device_id);

-- rollout_stages: each rollout has unique sequential stage
-- numbers
CREATE UNIQUE INDEX uidx_rollout_stages_rollout_stage ON
    rollout_stages (rollout_id, stage_number);

-- users: username must be unique among non-deleted users
CREATE UNIQUE INDEX uidx_users_username ON users (username)
    WHERE deleted_at IS NULL;

-- users: email must be unique among non-deleted users
CREATE UNIQUE INDEX uidx_users_email ON users (email) WHERE
    deleted_at IS NULL;
```

4.3 Foreign Key Indexes

Foreign key columns MUST be indexed to prevent sequential scans during CASCADE operations and to speed up JOIN queries.

```
-- devices → device_groups
CREATE INDEX idx_devices_group_id ON devices (group_id);

-- artifacts → users (uploaded_by)
CREATE INDEX idx_artifacts_uploaded_by ON artifacts
    (uploaded_by) WHERE deleted_at IS NULL;

-- artifact_validation_results → artifacts
CREATE INDEX idx_artifact_validation_artifact_id ON
    artifact_validation_results (artifact_id);

-- rollouts → artifacts
CREATE INDEX idx_rollouts_artifact_id ON rollouts
    (artifact_id);

-- rollouts → device_groups
CREATE INDEX idx_rollouts_target_group_id ON rollouts
    (target_group_id) WHERE target_group_id IS NOT
    NULL;

-- rollouts → users (created_by)
CREATE INDEX idx_rollouts_created_by ON rollouts
    (created_by) WHERE created_by IS NOT NULL;

-- rollout_stages → rollouts
CREATE INDEX idx_rollout_stages_rollout_id
    ON rollout_stages (rollout_id);
```

```

-- device_updates → devices
CREATE INDEX idx_device_updates_device_id ON device_updates
(device_id);

-- device_updates → rollouts
CREATE INDEX idx_device_updates_rollout_id
ON device_updates (rollout_id) WHERE rollout_id IS
NOT NULL;

-- device_updates → artifacts
CREATE INDEX idx_device_updates_artifact_id ON
device_updates (artifact_id);

-- audit_log → users
CREATE INDEX idx_audit_log_user_id ON audit_log (user_id)
WHERE user_id IS NOT NULL;

```

4.4 Composite Indexes for Common Query Patterns

These indexes target the most frequent and performance-critical queries identified in the application layer.

```

-- Device check-in: find device by device_id + verify it's
not deleted
-- Supports: UpdateService.CheckForUpdate()
CREATE INDEX idx_devices_device_id_active ON devices
(device_id, status, group_id)
WHERE deleted_at IS NULL;

-- Fleet overview: list devices by group with status
filtering
-- Supports: Dashboard fleet view, group-based device counts
CREATE INDEX idx_devices_group_status ON devices (group_id,
status)
WHERE deleted_at IS NULL;

-- Active rollouts: find active rollouts targeting a
specific group
-- Supports: UpdateService.CheckForUpdate() - rollout lookup
CREATE INDEX idx_rollouts_active_group ON rollouts
(target_group_id, status)
WHERE status IN ('active', 'paused');

-- Rollout status monitoring: rollouts by status for
dashboard
-- Supports: Dashboard rollout list, auto-advance scheduler
CREATE INDEX idx_rollouts_status ON rollouts (status)
WHERE status IN ('active', 'paused');

-- Device update tracking: latest update for a device
-- Supports: DeviceService.GetDeviceStatus()

```

```

CREATE INDEX idx_device_updates_device_created ON
    device_updates (device_id, created_at DESC);

-- Device update tracking: all updates for a rollout with
    status
-- Supports: RolloutService.getRolloutHealth(), progress
    calculation
CREATE INDEX idx_device_updates_rollout_status ON
    device_updates (rollout_id, status)
    WHERE rollout_id IS NOT NULL;

-- Artifact version lookup: find artifact by version +
    os_type
-- Supports: UpdateService version compatibility check
CREATE INDEX idx_artifacts_version_os
    ON artifacts (version, os_type, os_version)
    WHERE deleted_at IS NULL AND upload_status = 'ready';

-- Validation results by artifact with step ordering
-- Supports: ArtifactService validation audit trail
CREATE INDEX idx_artifact_validation_artifact_step ON
    artifact_validation_results (artifact_id,
        validation_step);

-- Rollout stages by rollout with ordering
-- Supports: RolloutService stage progression
CREATE INDEX idx_rollout_stages_rollout_stage_num ON
    rollout_stages (rollout_id, stage_number);

-- Audit log by resource type and ID
-- Supports: Resource history queries
CREATE INDEX idx_audit_log_resource ON audit_log
    (resource_type, resource_id);

-- Audit log by user for user activity audit
-- Supports: User activity reports
CREATE INDEX idx_audit_log_user_created ON audit_log
    (user_id, created_at DESC)
    WHERE user_id IS NOT NULL;

```

4.5 Partial Indexes

Partial indexes reduce index size and improve query performance by indexing only rows that match common query predicates.

```

-- Active (non-deleted) devices: most queries only need
    active devices
CREATE INDEX idx_devices_active ON devices (status,
    last_seen_at DESC)
    WHERE deleted_at IS NULL;

-- Devices currently updating: for monitoring in-flight
    updates

```

```

CREATE INDEX idx_devices_updating ON devices (id, device_id)
    WHERE status = 'updating' AND deleted_at IS NULL;

-- Offline devices (stale detection): for anomaly alerting
CREATE INDEX idx_devices_stale ON devices (last_seen_at,
    device_id)
    WHERE status = 'offline' AND deleted_at IS NULL;

-- Ready artifacts: most queries filter to ready status only
CREATE INDEX idx_artifacts_ready ON artifacts (version,
    os_type, created_at DESC)
    WHERE upload_status = 'ready' AND deleted_at IS NULL;

-- Failed validation results: for debugging failed uploads
CREATE INDEX idx_artifact_validation_failed ON
    artifact_validation_results (artifact_id,
    validation_step)
    WHERE status = 'failed';

-- Active rollouts: scheduler and dashboard queries
CREATE INDEX idx_rollouts_active ON rollouts (status,
    created_at DESC)
    WHERE status IN ('active', 'paused');

-- In-progress device updates: for monitoring active updates
CREATE INDEX idx_device_updates_in_progress ON
    device_updates (device_id, status)
    WHERE status IN ('pending', 'downloading', 'verifying',
    'installing', 'rebooting', 'committing');

-- Failed device updates: for failure analysis and rollback
    tracking
CREATE INDEX idx_device_updates_failed ON device_updates
    (artifact_id, error_code, created_at DESC)
    WHERE status = 'failed';

-- Pending rollout stages: for auto-advance scheduler
CREATE INDEX idx_rollout_stages_pending ON rollout_stages
    (rollout_id, stage_number)
    WHERE status IN ('pending', 'active');

```

4.6 GIN Indexes for JSONB Columns

JSONB columns that are queried using containment (@>) or existence (?) operators require GIN indexes for acceptable performance.

```

-- Device group filter rules: matching devices to groups
-- Supports: "find groups whose filter_rules match this
    device"
CREATE INDEX idx_device_groups_filter_rules ON
    device_groups USING GIN (filter_rules
    jsonb_path_ops);

```

```

-- Artifact hardware compatibility: checking if an artifact
  supports a device model
-- Supports: "find artifacts where hardware_compatibility @>
  '['rk3588_opi5max']'"
CREATE INDEX idx_artifacts_hw_compat ON artifacts USING GIN
  (hardware_compatibility jsonb_path_ops)
WHERE deleted_at IS NULL;

-- Artifact payload metadata: querying by update type,
  source version
-- Supports: "find full-update artifacts for version >=
  15.0.0"
CREATE INDEX idx_artifacts_payload_metadata ON artifacts
USING GIN (payload_metadata)
WHERE deleted_at IS NULL;

-- Telemetry event payload: searching for specific error
  patterns
-- Supports: "find telemetry where payload contains
  error_code INSTALL_FAILED"
CREATE INDEX idx_telemetry_events_payload ON
  telemetry_events USING GIN (payload
  jsonb_path_ops);

-- Validation result details: searching for specific failure
  patterns
-- Supports: "find validation failures with specific error
  messages"
CREATE INDEX idx_artifact_validation_details ON
  artifact_validation_results USING GIN (details
  jsonb_path_ops);

-- Audit log details: searching for specific action
  attributes
CREATE INDEX idx_audit_log_details ON audit_log USING GIN
  (details jsonb_path_ops);

```

4.7 Telemetry-Specific Indexes

The telemetry_events table requires special index treatment due to its partitioned nature and high-volume time-series access patterns.

```

-- These indexes must be created on each partition (see
  Section 7 for automation)
-- Device timeline: all events for a device in chronological
  order
CREATE INDEX idx_telemetry_events_device_time ON
  telemetry_events (device_id, created_at DESC);

-- Event type filtering: find all events of a specific type
CREATE INDEX idx_telemetry_events_type_time ON
  telemetry_events (event_type, created_at DESC);

```

```

-- Device + event type: targeted queries for specific device
events
CREATE INDEX idx_telemetry_events_device_type ON
    telemetry_events (device_id, event_type,
        created_at DESC);

-- Failure events: for anomaly detection
CREATE INDEX idx_telemetry_events_failures ON
    telemetry_events (event_type, created_at DESC)
    WHERE event_type IN ('failure', 'rollback');

-- Recent events: for dashboard real-time view (last 24
hours)
CREATE INDEX idx_telemetry_events_recent
    ON telemetry_events (created_at DESC)
    WHERE created_at > NOW() - INTERVAL '24 hours';

```

4.8 Complete Index Summary

Index Name	Table	Type	C
uidx_devices_device_id	devices	Unique B-tree	de
uidx_users_username	users	Unique B-tree	us
uidx_users_email	users	Unique B-tree	en
uidx_rollout_stages_rollout_stage	rollout_stages	Unique B-tree	ro st
idx_devices_group_id	devices	B-tree	gr
idx_devices_device_id_active	devices	B-tree	de gr
idx_devices_group_status	devices	B-tree	gr
idx_devices_active	devices	B-tree	st D.
idx_devices_updating	devices	B-tree	id
idx_devices_stale	devices	B-tree	la
idx_artifacts_uploaded_by	artifacts	B-tree	up
idx_artifacts_version_os	artifacts	B-tree	ve os

Index Name	Table	Type	Comment
idx_artifacts_ready	artifacts	B-tree	used for verification
idx_artifacts_hw_compat	artifacts	GIN	hardware compatibility
idx_artifacts_payload_metadata	artifacts	GIN	payload metadata
idx_artifact_validation_artifact_id	artifact_validation_results	B-tree	artifact ID
idx_artifact_validation_artifact_step	artifact_validation_results	B-tree	artifact step
idx_artifact_validation_failed	artifact_validation_results	B-tree	failed validation
idx_artifact_validation_details	artifact_validation_results	GIN	validation details
idx_rollouts_artifact_id	rollouts	B-tree	artifact ID
idx_rollouts_active_group	rollouts	B-tree	active group
idx_rollouts_status	rollouts	B-tree	status
idx_rollouts_active	rollouts	B-tree	status: Done
idx_rollout_stages_rollout_id	rollout_stages	B-tree	rollout ID
idx_rollout_stages_pending	rollout_stages	B-tree	rollout status
idx_device_updates_device_id	device_updates	B-tree	device ID
idx_device_updates_device_created	device_updates	B-tree	device ID: Done
idx_device_updates_rollout_status	device_updates	B-tree	rollout status
idx_device_updates_in_progress	device_updates	B-tree	device ID

Index Name	Table	Type	C
idx_device_updates_failed	device_updates	B-tree	an cr
idx_telemetry_events_device_time	telemetry_events	B-tree	de D
idx_telemetry_events_type_time	telemetry_events	B-tree	ev D
idx_telemetry_events_payload	telemetry_events	GIN	pa
idx_audit_log_resource	audit_log	B-tree	re re
idx_audit_log_user_created	audit_log	B-tree	us D
idx_audit_log_details	audit_log	GIN	de

5. Migration Strategy

5.1 Tooling

Helix OTA uses [golang-migrate](#) for database schema versioning. This tool provides:

- **Sequential version numbering** — Ensures deterministic migration order.
- **Up and down migrations** — Every change is reversible.
- **Lock-based execution** — Prevents concurrent migration execution in multi-instance deployments.
- **CLI + Go API** — Migrations can be run from CI/CD pipelines or embedded in the application binary.

5.2 Migration File Convention

```
migrations/
├── 000001_create_enum_types.up.sql
├── 000001_create_enum_types.down.sql
├── 000002_create_device_groups.up.sql
├── 000002_create_device_groups.down.sql
├── 000003_create_users.up.sql
├── 000003_create_users.down.sql
├── 000004_create_devices.up.sql
├── 000004_create_devices.down.sql
```

```

└─ 000005_create_artifacts.up.sql
└─ 000005_create_artifacts.down.sql
└─ 000006_create_artifact_validation_results.up.sql
└─ 000006_create_artifact_validation_results.down.sql
└─ 000007_create_rollouts.up.sql
└─ 000007_create_rollouts.down.sql
└─ 000008_create_rollout_stages.up.sql
└─ 000008_create_rollout_stages.down.sql
└─ 000009_create_device_updates.up.sql
└─ 000009_create_device_updates.down.sql
└─ 000010_create_telemetry_events.up.sql
└─ 000010_create_telemetry_events.down.sql
└─ 000011_create_audit_log.up.sql
└─ 000011_create_audit_log.down.sql
└─ 000012_create_indexes.up.sql
└─ 000012_create_indexes.down.sql
└─ 000013_create_triggers.up.sql
└─ 000013_create_triggers.down.sql
└─ 000014_create_telemetry_partitions.up.sql
└─ 000014_create_telemetry_partitions.down.sql
└─ 000015_seed_initial_data.up.sql
    000015_seed_initial_data.down.sql

```

5.3 Migration Execution

Development (from project root)

```

migrate -path migrations -database "postgres://
    helix:helix@localhost:5432/helix_ota?
    sslmode=disable" up

```

Production (via CI/CD pipeline)

```

migrate -path migrations -database "$DATABASE_URL" up -n 1
    # Apply one migration at a time

```

Rollback last migration

```

migrate -path migrations -database "$DATABASE_URL" down 1

```

Force version (emergency recovery)

```

migrate -path migrations -database "$DATABASE_URL" force 13

```

5.4 Migration Best Practices

1. **Always provide down migrations** — Every up.sql MUST have a corresponding down.sql that reverses the change.
2. **No data loss in down migrations** — Down migrations should be safe to execute; prefer dropping constraints/indexes over dropping columns with data.
3. **Single concern per migration** — Each migration addresses one logical change (create table, add index, alter column).

4. **Idempotent where possible** — Use IF EXISTS / IF NOT EXISTS for defensive migration writing.
5. **No application logic in migrations** — Migrations contain only DDL/DML; business logic belongs in the application layer.
6. **Test up and down** — CI pipeline applies all up migrations, then all down migrations, then up again to verify reversibility.
7. **Large data migrations** — For migrations that modify large volumes of data (e.g., backfill columns), use batched updates with LIMIT and OFFSET to avoid long-running locks.

5.5 Seed Data

The initial seed migration creates the default device group and admin user:

```
-- 000015_seed_initial_data.up.sql

-- Default device group for RK3588 Orange Pi 5 Max
INSERT INTO device_groups (id, name, description,
                           filter_rules)
VALUES (
    'a1b2c3d4-e5f6-7890-abcd-ef1234567890',
    'RK3588 Orange Pi 5 Max',
    'Default group for all RK3588 Orange Pi 5 Max devices
    running Android 15',
    '{"hardware_model": "rk3588_opi5max", "os_type":
    "android", "os_version": {"gte": "15.0.0"}}'
);

-- Default admin user (password: change_me_immediately)
INSERT INTO users (id, username, email, password_hash, role)
VALUES (
    'b2c3d4e5-f6a7-8901-bcde-f12345678901',
    'admin',
    'admin@helix-ota.local',
    '$2a$12$LJ3m4ys3Lk0TSwMCfVSLn0XLwHb0NbKxHP0R31EVbwMGF7cGfSaKa',
    'admin'
);
```

6. Sample Queries

6.1 Device Check for Update

This is the most latency-sensitive query in the system. It is called by every device on every check-in (default: every 4 hours). The query must complete in under 10ms at P99.

```
-- Find an available update for a specific device
EXPLAIN ANALYZE
SELECT
    a.id          AS artifact_id,
    a.version     AS target_version,
    a.filename,
    a.file_size,
    a.file_hash_sha256,
    a.signature_hash,
    a.signature_algorithm,
    r.id          AS rollout_id,
    r.strategy,
    r.current_percentage
FROM devices d
INNER JOIN rollouts r
    ON r.target_group_id = d.group_id
    AND r.status = 'active'
    AND r.current_percentage > 0
INNER JOIN artifacts a
    ON a.id = r.artifact_id
    AND a.upload_status = 'ready'
    AND a.deleted_at IS NULL
WHERE d.device_id = 'DEV-OPi5M-00142'
    AND d.deleted_at IS NULL
    AND (
        -- Deterministic cohort check: device is in the rollout cohort
        -- Application computes: cohort = fnv32(device_id) % 100
        -- If cohort < current_percentage, device gets the update
        TRUE -- Placeholder; actual cohort check is in application layer
    )
    AND a.hardware_compatibility @>
        to_jsonb(d.hardware_model)::jsonb
ORDER BY r.created_at DESC
LIMIT 1;
```

Expected plan: Index Scan on idx_devices_device_id_active
→ Index Scan on idx_rollouts_active_group → Index Scan on
pk_artifacts → Filter with GIN index
idx_artifacts_hw_compat.

Optimization notes: - The cohort calculation (`fnnv32(device_id) % 100 < current_percentage`) is performed in the application layer, not in SQL, to keep the query simple and cacheable. - The result is cached in Redis with a 60-second TTL. - The hardware_compatibility @> containment check uses the GIN index on hardware_compatibility.

6.2 Rollout Progress Calculation

Computes the aggregate progress and health of a rollout for the dashboard. This query runs every 15 seconds for each active rollout displayed on the dashboard.

```
-- Calculate rollout progress and health metrics
EXPLAIN ANALYZE
SELECT
    r.id AS rollout_id,
    r.name AS rollout_name,
    r.current_percentage,
    r.target_percentage,
    r.status AS rollout_status,
    COUNT(du.id) AS total_updates,
    COUNT(du.id) FILTER (WHERE du.status = 'succeeded') AS
        succeeded_count,
    COUNT(du.id) FILTER (WHERE du.status = 'failed') AS
        failed_count,
    COUNT(du.id) FILTER (WHERE du.status = 'rolled_back')
        AS rolled_back_count,
    COUNT(du.id) FILTER (WHERE du.status IN ('pending',
        'downloading', 'verifying',
        'installing', 'rebooting', 'committing')) AS
        in_progress_count,
    COUNT(du.id) FILTER (WHERE du.status = 'pending') AS
        pending_count,
    ROUND(
        COUNT(du.id) FILTER (WHERE du.status =
            'succeeded')::FLOAT
        / NULLIF(COUNT(du.id), 0) * 100, 2
    ) AS success_rate_pct,
    ROUND(
        COUNT(du.id) FILTER (WHERE du.status =
            'failed')::FLOAT
        / NULLIF(COUNT(du.id), 0) * 100, 2
    ) AS failure_rate_pct,
    AVG(
        EXTRACT(EPOCH FROM (du.completed_at -
            du.started_at))
    ) FILTER (WHERE du.status = 'succeeded') AS
        avg_duration_seconds
FROM rollouts r
LEFT JOIN device_updates du ON du.rollout_id = r.id
```

```
WHERE r.id = '550e8400-e29b-41d4-a716-446655440000'
GROUP BY r.id, r.name, r.current_percentage,
         r.target_percentage, r.status;
```

Expected plan: Index Scan on rollouts_pkey → Index Scan on idx_device_updates_rollout_status → Aggregate.

Optimization notes: - This query benefits heavily from the idx_device_updates_rollout_status composite index. - The FILTER (WHERE ...) aggregate syntax avoids the need for multiple subqueries. - Results are cached in Redis for 60 seconds by the TelemetryService. - NULLIF(COUNT(du.id), 0) prevents division-by-zero for rollouts with no assigned updates yet.

6.3 Fleet Health Overview

Dashboard landing page query: summarizes the entire fleet's health. Must be fast enough for real-time dashboard rendering.

-- Fleet health overview

EXPLAIN ANALYZE

SELECT

```
    dg.id AS group_id,
    dg.name AS group_name,
    COUNT(d.id) AS total_devices,
    COUNT(d.id) FILTER (WHERE d.status = 'online') AS
        online_count,
    COUNT(d.id) FILTER (WHERE d.status = 'offline') AS
        offline_count,
    COUNT(d.id) FILTER (WHERE d.status = 'updating') AS
        updating_count,
    COUNT(d.id) FILTER (WHERE d.status = 'error') AS
        error_count,
    COUNT(d.id) FILTER (WHERE d.last_seen_at > NOW() -
        INTERVAL '1 hour') AS recently_active_count,
    COUNT(d.id) FILTER (WHERE d.last_seen_at < NOW() -
        INTERVAL '24 hours') AS stale_count,
    MAX(d.last_seen_at) AS last_seen_max
```

FROM device_groups dg

LEFT JOIN devices d ON d.group_id = dg.id AND d.deleted_at
IS NULL

GROUP BY dg.id, dg.name

ORDER BY dg.name;

Expected plan: Seq Scan on device_groups → Index Scan on idx_devices_group_status → Hash Aggregate.

Optimization notes: - The `idx_devices_group_status` composite index provides efficient grouping and status filtering. - For fleets exceeding 100,000 devices, consider materializing this as a refreshed materialized view (refreshed every 30 seconds) rather than a live query.

6.4 Device Update History

Returns the complete update history for a single device, including rollout information and timing details.

-- Device update history with rollout context

EXPLAIN ANALYZE

SELECT

```
    du.id AS update_id,  
    du.status AS update_status,  
    du.created_at AS assigned_at,  
    du.started_at,  
    du.completed_at,  
    du.download_progress,  
    du.error_code,  
    du.error_message,  
    a.version AS artifact_version,  
    a.filename AS artifact_filename,  
    r.name AS rollout_name,  
    r.strategy AS rollout_strategy,  
    rs.stage_number,  
    rs.status AS stage_status
```

FROM device_updates du

INNER JOIN artifacts a **ON** a.id = du.artifact_id

LEFT JOIN rollouts r **ON** r.id = du.rollout_id

LEFT JOIN rollout_stages rs **ON** rs.rollout_id = r.id

WHERE du.device_id = (

```
    SELECT id FROM devices WHERE device_id = 'DEV-  
        OPi5M-00142' AND deleted_at IS NULL
```

)

ORDER BY du.created_at **DESC**

LIMIT 50;

Expected plan: Index Scan on devices_pkey (subquery) → Index Scan on idx_device_updates_device_created → Nested Loop Joins.

Optimization notes: - The `idx_device_updates_device_created` index provides efficient time-ordered retrieval. - The `LIMIT 50` prevents unbounded result sets for devices with long update histories. - The subquery for `device_id` to `id` resolution could be cached in the application layer.

6.5 Failure Analysis

Identifies patterns in update failures across the fleet. Used by the anomaly detection system and for post-mortem analysis.

```
-- Top failure patterns across the fleet (last 7 days)
EXPLAIN ANALYZE
SELECT
    du.error_code,
    du.status AS update_status,
    a.version AS artifact_version,
    a.hardware_compatibility,
    COUNT(*) AS failure_count,
    COUNT(DISTINCT du.device_id) AS affected_devices,
    MIN(du.created_at) AS first_occurrence,
    MAX(du.created_at) AS last_occurrence,
    jsonb_agg(DISTINCT du.error_message) AS error_messages
FROM device_updates du
INNER JOIN artifacts a ON a.id = du.artifact_id
WHERE du.status IN ('failed', 'rolled_back')
    AND du.created_at > NOW() - INTERVAL '7 days'
GROUP BY du.error_code, du.status, a.version,
    a.hardware_compatibility
ORDER BY failure_count DESC
LIMIT 20;
```

Expected plan: Index Scan on idx_device_updates_failed → Index Scan on pk_artifacts → Hash Aggregate → Sort.

Optimization notes: - The partial index idx_device_updates_failed makes this query extremely efficient by scanning only failed updates. - jsonb_agg(DISTINCT du.error_message) aggregates unique error messages without inflating row counts. - The 7-day window prevents the query from scanning the entire history.

Related query: Failed devices in a specific rollout:

```
-- Devices that failed during a specific rollout
SELECT
    d.device_id,
    d.hardware_model,
    du.status AS update_status,
    du.error_code,
    du.error_message,
    du.created_at AS update_assigned_at,
    du.completed_at AS failure_time,
    EXTRACT(EPOCH FROM (du.completed_at - du.started_at))
        AS seconds_before_failure
FROM device_updates du
INNER JOIN devices d ON d.id = du.device_id
WHERE du.rollout_id = '550e8400-e29b-41d4-a716-446655440000'
```



```
    AND du.status IN ('failed', 'rolled_back')
ORDER BY du.completed_at DESC;
```

7. Partitioning Strategy

7.1 Rationale

The telemetry_events table is the highest-volume table in the system. With devices reporting events at every stage of the update lifecycle (plus periodic progress updates), a fleet of 10,000 devices generating an average of 5 events per day produces approximately 50,000 rows/day or 1.5 million rows/month. At scale (100,000 devices), this reaches 15 million rows/month.

Partitioning by month provides:

1. **Query performance** — Queries with date ranges scan only relevant partitions.
2. **Maintenance efficiency** — VACUUM, ANALYZE, and REINDEX operate on individual partitions.
3. **Data lifecycle** — Old partitions can be detached and archived/dropped without affecting active data.
4. **Index size** — Each partition has its own indexes, keeping B-tree depth shallow.

7.2 Declarative Partitioning Configuration

The parent table is defined with PARTITION BY RANGE (created_at):

```
CREATE TABLE telemetry_events (  
    id          UUID          NOT NULL,  
    device_id   UUID          NOT NULL,  
    event_type  telemetry_event_type_enum NOT NULL,  
    payload     JSONB         DEFAULT '{}',  
    created_at  TIMESTAMPTZ   NOT NULL  
        DEFAULT NOW(),  
  
    CONSTRAINT pk_telemetry_events PRIMARY KEY (id,  
        created_at),  
    CONSTRAINT fk_telemetry_events_device FOREIGN KEY  
        (device_id) REFERENCES devices(id) ON DELETE  
        CASCADE  
) PARTITION BY RANGE (created_at);
```

7.3 Monthly Partition Template

Each partition covers one calendar month. The naming convention is `telemetry_events_YYYYMM`.

```
-- Partition creation template
CREATE TABLE telemetry_events_202603 PARTITION OF
    telemetry_events
    FOR VALUES FROM ('2026-03-01') TO ('2026-04-01');

CREATE TABLE telemetry_events_202604 PARTITION OF
    telemetry_events
    FOR VALUES FROM ('2026-04-01') TO ('2026-05-01');

CREATE TABLE telemetry_events_202605 PARTITION OF
    telemetry_events
    FOR VALUES FROM ('2026-05-01') TO ('2026-06-01');

-- Default partition (catches any out-of-range data;
    prevents insertion errors)
CREATE TABLE telemetry_events_default PARTITION OF
    telemetry_events DEFAULT;
```

7.4 Partition Indexes

Each partition inherits the parent table's constraint definitions but requires its own indexes. These must be created on each partition:

```
-- Applied to each monthly partition
CREATE INDEX idx_telemetry_events_202603_device_time
    ON telemetry_events_202603 (device_id, created_at DESC);

CREATE INDEX idx_telemetry_events_202603_type_time
    ON telemetry_events_202603 (event_type, created_at
    DESC);

CREATE INDEX idx_telemetry_events_202603_device_type
    ON telemetry_events_202603 (device_id, event_type,
    created_at DESC);

CREATE INDEX idx_telemetry_events_202603_payload
    ON telemetry_events_202603 USING GIN (payload
    jsonb_path_ops);
```

7.5 Automated Partition Management

A `pg_cron` job (or application-scheduled job) creates partitions in advance and detaches old partitions:

```
-- Enable pg_cron extension
CREATE EXTENSION IF NOT EXISTS pg_cron;
```

```

-- Scheduled job: create next month's partition on the 25th
  of each month
SELECT cron.schedule(
  'create-telemetry-partition',
  '0 2 25 * *', -- 02:00 on the 25th of each month
  $$
DO $func__
DECLARE
  next_month DATE := date_trunc('month', NOW()) +
    INTERVAL '2 months';
  part_name TEXT := 'telemetry_events_' ||
    to_char(next_month, 'YYYYMM');
  start_date DATE := next_month;
  end_date DATE := next_month + INTERVAL '1 month';
BEGIN
  EXECUTE format(
    'CREATE TABLE %I PARTITION OF telemetry_events
    FOR VALUES FROM (%L) TO (%L)',
    part_name, start_date, end_date
  );
  -- Create partition-specific indexes
  EXECUTE format(
    'CREATE INDEX idx_%s_device_time ON %s
    (device_id, created_at DESC)',
    part_name, part_name
  );
  EXECUTE format(
    'CREATE INDEX idx_%s_type_time ON %s
    (event_type, created_at DESC)',
    part_name, part_name
  );
  EXECUTE format(
    'CREATE INDEX idx_%s_device_type ON %s
    (device_id, event_type, created_at DESC)',
    part_name, part_name
  );
  EXECUTE format(
    'CREATE INDEX idx_%s_payload ON %s USING GIN
    (payload jsonb_path_ops)',
    part_name, part_name
  );
  RAISE NOTICE 'Created partition % for range [% to
    %]', part_name, start_date, end_date;
END;
$func__;
$$
);

-- Scheduled job: detach partitions older than 90 days (run
  daily at 03:00)
SELECT cron.schedule(
  'detach-old-telemetry-partitions',

```

```

'0 3 * * *',
$$
DO $func__
DECLARE
    part RECORD;
BEGIN
    FOR part IN
        SELECT child.relname AS partition_name,
            pg_get_expr(child.relpartbound,
child.oid) AS partition_expr
        FROM pg_inherits
        JOIN pg_class parent ON pg_inherits.inhparent =
parent.oid
        JOIN pg_class child ON pg_inherits.inhrelid =
child.oid
        WHERE parent.relname = 'telemetry_events'
        AND child.relname !=
'telemetry_events_default'
    LOOP
        -- Extract date from partition name
        (telemetry_events_YYYYMM)
        IF part.partition_name ~
'telemetry_events_\d{6}$' THEN
            DECLARE
                part_date DATE := to_date(
                    substring(part.partition_name from
'(\d{6})$'),
                    'YYYYMM'
                );
            BEGIN
                IF part_date < date_trunc('month',
NOW()) - INTERVAL '90 days') THEN
                    EXECUTE format('ALTER TABLE
telemetry_events DETACH PARTITION %I',
part.partition_name);
                    RAISE NOTICE 'Detached old
partition: %', part.partition_name;

                    -- Optionally: DROP TABLE or move to cold storage
                END IF;
            END;
        END IF;
    END LOOP;
END;
$func__;
$$
);

```

7.6 Partition Pruning Verification

PostgreSQL 16 supports partition pruning at both plan time (static) and execution time (dynamic). Verify pruning is working:

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT COUNT(*)
FROM telemetry_events
WHERE device_id = 'a1b2c3d4-e5f6-7890-abcd-ef1234567890'
      AND created_at >= '2026-03-01'
      AND created_at < '2026-04-01';
```

Expected output should show:

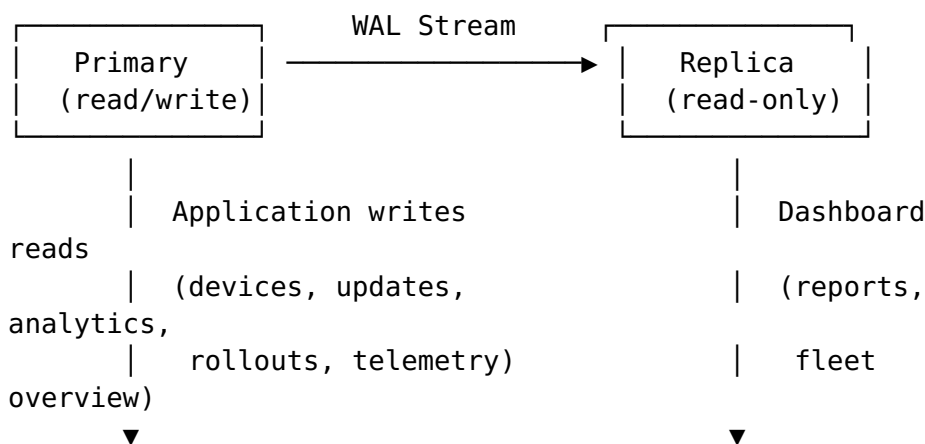
```
-> Index Only Scan using
idx_telemetry_events_202603_device_type on
telemetry_events_202603
    Index Cond: (device_id = '...' AND created_at >=
'2026-03-01' AND created_at < '2026-04-01')
```

Only the `telemetry_events_202603` partition should be scanned. If multiple partitions appear, check that `constraint_exclusion` is set to `partition` (default in PG 16).

8. Replication & Backup

8.1 Streaming Replication

Helix OTA uses PostgreSQL native streaming replication for high availability. The production deployment uses a primary-replica topology:



Primary configuration (postgresql.conf):

```
# Replication
wal_level = replica
```

```
max_wal_senders = 5
wal_keep_size = 256MB
synchronous_commit = on
synchronous_standby_names = 'helix_replica_1'
```

Performance

```
shared_buffers = 4GB
effective_cache_size = 12GB
work_mem = 64MB
maintenance_work_mem = 512MB
```

WAL Tuning

```
max_wal_size = 2GB
min_wal_size = 512MB
checkpoint_completion_target = 0.9
```

Replica configuration (postgresql.conf):

Replication

```
hot_standby = on
hot_standby_feedback = on
max_standby_streaming_delay = 30s
```

Connection

```
listen_addresses = '*'
```

Performance (same as primary)

```
shared_buffers = 4GB
effective_cache_size = 12GB
```

Replication setup (pg_hba.conf on primary):

Allow replication connections from replica

```
host      replication      helix_replica    10.0.1.20/32
          scram-sha-256
```

Replica initialization:

On the replica server

```
pg_basebackup -h 10.0.1.10 -U helix_replica -D /var/lib/
postgresql/16/main -Fp -Xs -P -R
systemctl start postgresql@16-main
```

8.2 Connection Routing

The application layer routes queries based on operation type:

Operation Type	Target	Rationale
Device check-in / update check	Primary	Low-latency write + read

Operation Type	Target	Rationale
Telemetry ingestion (batch INSERT)	Primary	Write operation
Rollout management	Primary	Write operation
Dashboard analytics	Replica	Read-heavy, tolerates slight staleness
Fleet overview	Replica	Read-heavy
Audit log queries	Replica	Read-only historical queries

Go application routing example:

```

type DBCluster struct {
    primary *pgxpool.Pool
    replica *pgxpool.Pool
}

func (c *DBCluster) WriteConn() *pgxpool.Pool { return c.primary }
func (c *DBCluster) ReadConn() *pgxpool.Pool { return c.replica }

```

8.3 Point-in-Time Recovery (PITR)

PostgreSQL's WAL archiving enables point-in-time recovery, allowing the database to be restored to any arbitrary point in time within the retention window.

WAL archive configuration (postgresql.conf):

```

# WAL Archiving
archive_mode = on
archive_command = 'aws s3 cp %p s3://helix-ota-wal-archive/%f'
archive_timeout = 300 # Force archive every 5 minutes even if WAL segment not full

```

Base backup scheduling:

```

# Daily base backup (run via cron at 02:00)
0 2 * * * pg_basebackup -D /backup/base/$(date +%Y%m%d) -Ft -z -P
0 3 * * * aws s3 sync /backup/base/$(date +%Y%m%d) s3://helix-ota-base-backups/$(date +%Y%m%d)/

```

Recovery procedure:

```

# 1. Stop PostgreSQL
systemctl stop postgresql@16-main

```

```
# 2. Restore base backup
tar -xzf /backup/base/20260305/base.tar.gz -C /var/lib/
    postgresql/16/main

# 3. Create recovery configuration
cat > /var/lib/postgresql/16/main/recovery.signal <<EOF
restore_command = 'aws s3 cp s3://helix-ota-wal-archive/%f
    %p'

recovery_target_time = '2026-03-05 14:30:00 UTC'
recovery_target_action = 'promote'
EOF

# 4. Start PostgreSQL (begins recovery)
systemctl start postgresql@16-main

# 5. Verify recovery
psql -c "SELECT pg_is_in_recovery();" # Should return 'f'
    (false = promoted)
```

8.4 Backup Retention Policy

Backup Type	Retention	Storage	Purpose
WAL archives	30 days	S3 (Standard)	Point-in-time recovery within 30 days
Daily base backups	90 days	S3 (Standard)	Full database restore
Weekly base backups	1 year	S3 (Glacier)	Compliance / disaster recovery
Monthly base backups	3 years	S3 (Glacier Deep Archive)	Regulatory compliance

8.5 Failover Procedure

In the event of primary failure, the replica is promoted:

```
# On the replica server
# 1. Verify replication lag is acceptable
psql -c "SELECT NOW() - pg_last_xact_replay_timestamp() AS
    replication_lag;"
# Should be < 5 seconds

# 2. Promote replica to primary
pg_ctl promote -D /var/lib/postgresql/16/main
```


- # 3. Update application configuration to point writes to the new primary
- # 4. Update DNS or load balancer to redirect traffic
- # 5. Set up a new replica from the promoted primary

8.6 Data Integrity Verification

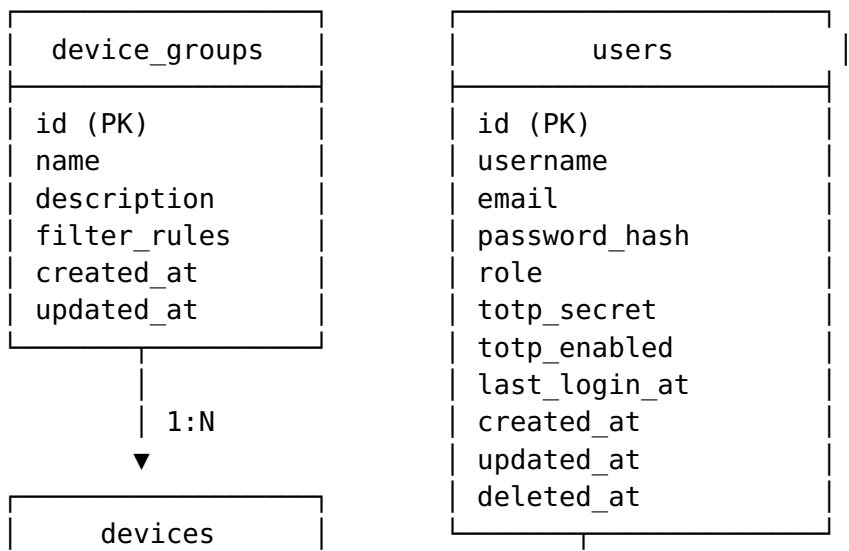
```
-- Verify data integrity across primary and replica
-- Run on replica; should match primary results

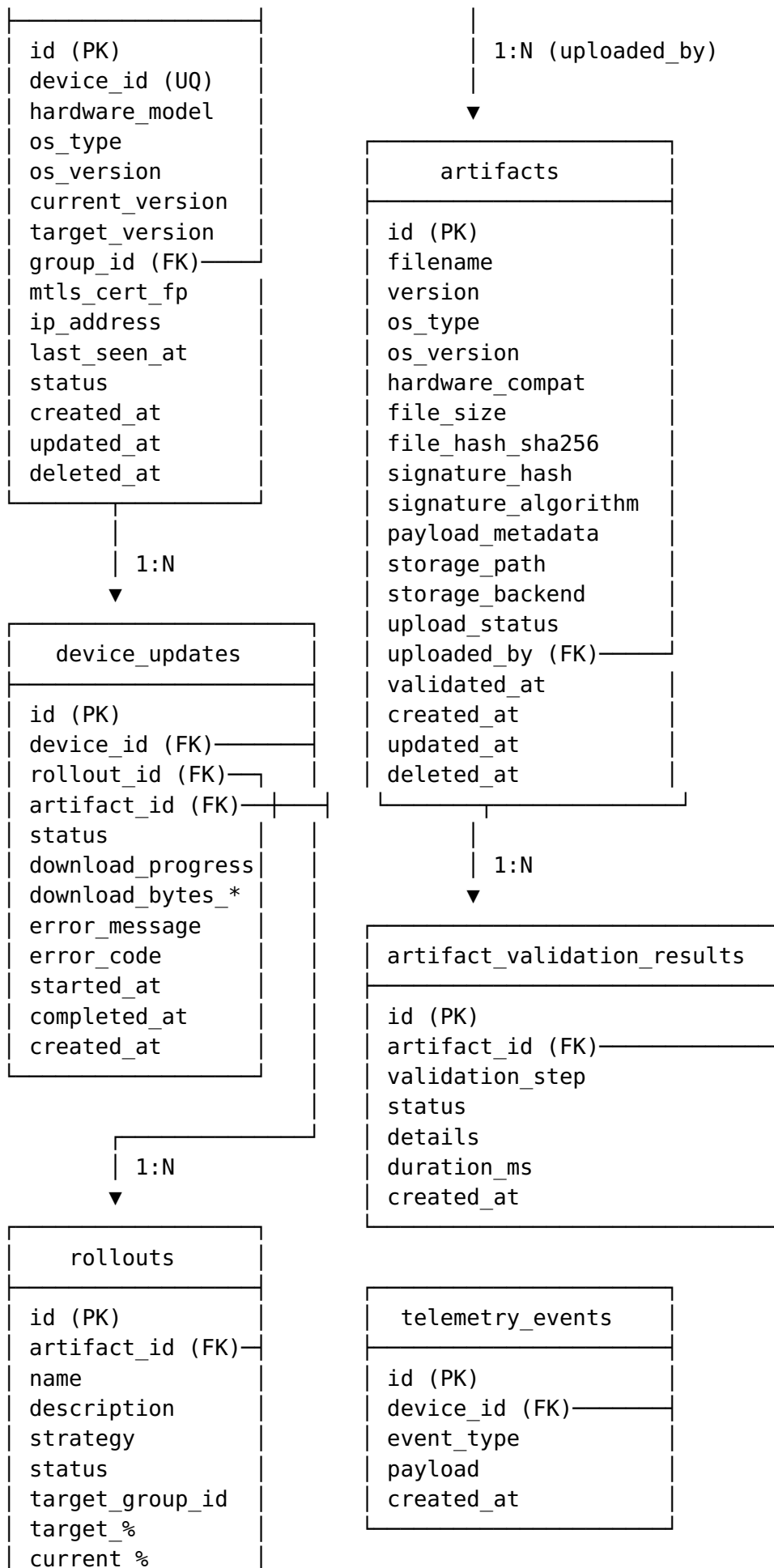
-- Row counts for key tables (sample)
SELECT 'devices' AS table_name,
       COUNT(*) AS row_count,
       COUNT(*) FILTER (WHERE deleted_at IS NULL) AS
       active_count
FROM devices;

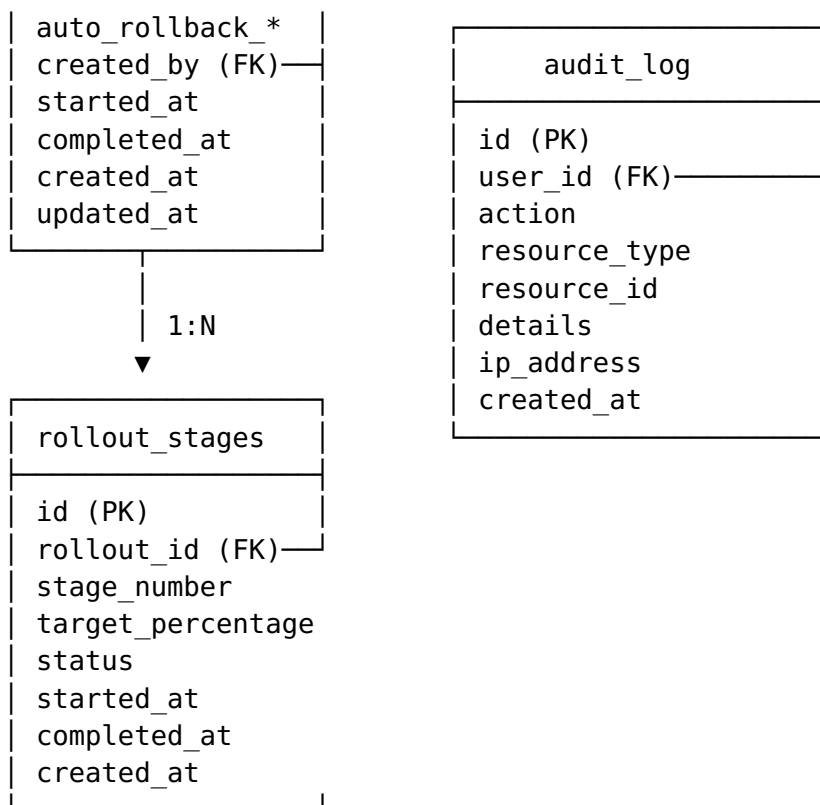
SELECT 'device_updates' AS table_name,
       COUNT(*) AS row_count,
       COUNT(*) FILTER (WHERE status = 'succeeded') AS
       succeeded_count,
       COUNT(*) FILTER (WHERE status = 'failed') AS
       failed_count
FROM device_updates;

SELECT 'telemetry_events' AS table_name,
       COUNT(*) AS row_count
FROM telemetry_events
WHERE created_at >= date_trunc('month', NOW());
```

Appendix A: Entity-Relationship Diagram







Appendix B: Table Size Estimates

Estimated table sizes for a fleet of **10,000 devices** over **1 year** of operation:

Table	Row Count (1 yr)	Avg Row Size	Total Size	Index Size	Total
devices	10,000	500 B	5 MB	8 MB	13 MB
device_groups	5-20	1 KB	20 KB	32 KB	52 KB
users	10-50	400 B	20 KB	40 KB	60 KB
artifacts	50-200	1 KB	200 KB	500 KB	700 KB
artifact_validation_results	200-800	800 B	640 KB	1 MB	1.6 MB
rollouts	20-100	600 B	60 KB	200 KB	260 KB
rollout_stages	60-400	200 B	80 KB	200 KB	280 KB
device_updates	10,000-50,000	500 B	25 MB	40 MB	65 MB

Table	Row Count (1 yr)	Avg Row Size	Total Size	Index Size	Total
telemetry_events	5M-18M	600 B	3-11 GB	2-6 GB	5-17 GB
audit_log	5,000-20,000	500 B	10 MB	15 MB	25 MB

Key observation: telemetry_events dominates storage by 2-3 orders of magnitude. The monthly partitioning strategy is essential for managing this table’s size and performance characteristics.